
title: "Neural Networks Part 8: Image Classification with Convolutional Neural Networks (CNNs) - YouTube"

description: Enjoy the videos and music that you love, upload original content and share it all with friends, family and the world on YouTube.

author: YouTube

source: <https://www.youtube.com/watch?v=HGwBXDKFk9I>

created: "2026-08-17"

tags:

- hover-notes
 - youtube
-

Image Classification with Convolutional Neural Networks

- Convolutional Neural Networks (CNNs) are used for tasks such as image classification

Prerequisites

- Assumed familiarity with several core neural network concepts:
 - The main ideas behind Neural Networks
 - Backpropagation
 - ReLU activation function
 - Neural networks with multiple inputs and outputs

Tic-Tac-Toe Example

- Introduction of a character, StatSquatch, who wants to play tic-tac-toe against a computer
 - StatSquatch has difficulty remembering if they started with the letter "X"

Using CNNs for Character Recognition

- StatSquatch needs the computer to identify if they drew an "X" or an "O"
- A Convolutional Neural Network can be used to figure this out
 - This demonstrates a practical application of CNNs for image classification

Digital Image Representation

- Images are composed of a grid of individual pixels
- When zooming in on a character (like "O" or "X"), the underlying pixel structure becomes visible
- Each pixel is represented by a numerical value to indicate its color:
 - 0 represents a white pixel
 - 1 represents a black pixel

Character	Zoomed View (Pixel Grid)
Letter "O"	A ring of 1s (black) surrounding a center of 0s (white)
Letter "X"	A diagonal pattern of 1s (black) on a background of 0s (white)

Classifying the Letter "O" with a Standard Neural Network

- Because the image of the letter "O" is very small (6 pixels wide by 6 pixels tall), a standard neural network can be used for classification
- The 2D grid of pixels is converted into a single column of input nodes
 - A 6×6 grid results in a single column of 36 input nodes
- These input nodes are then connected to a hidden layer

Input Layer

0

0

1

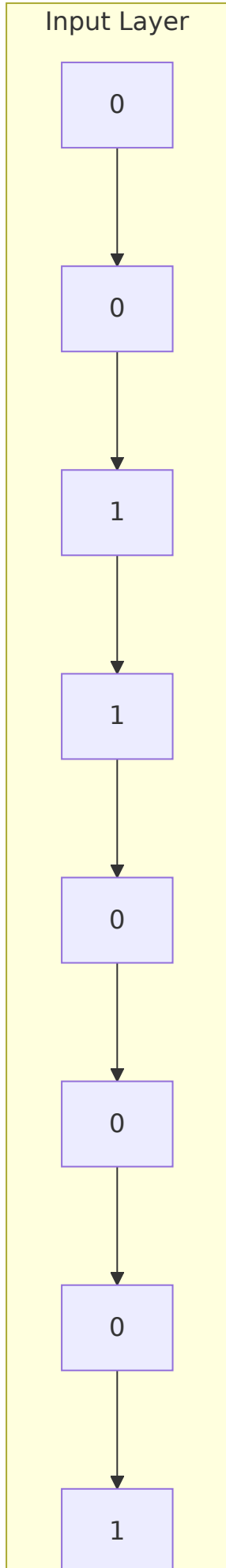
1

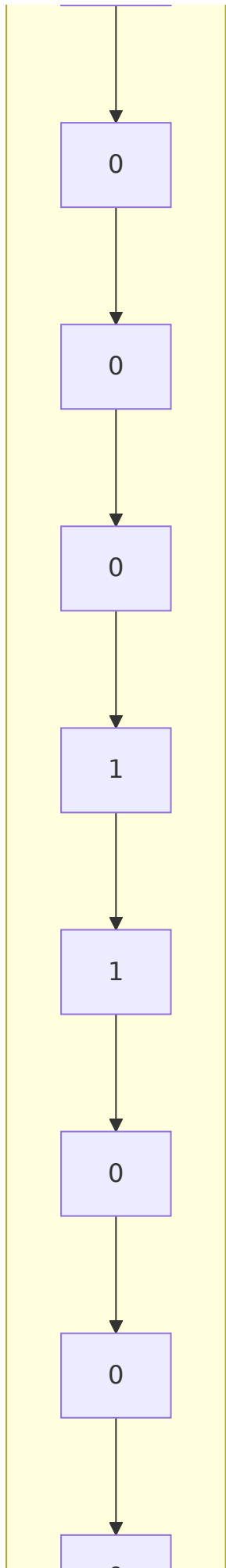
0

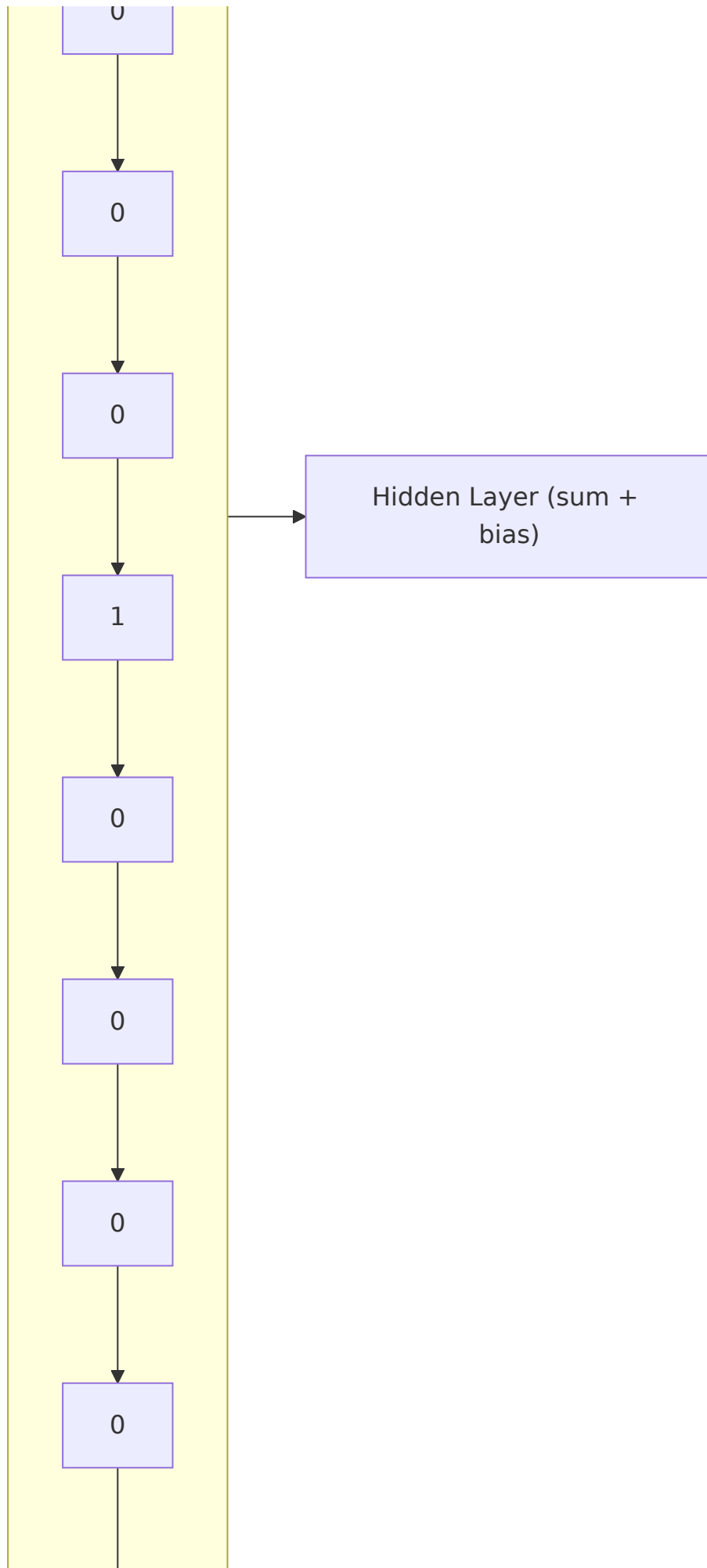
0

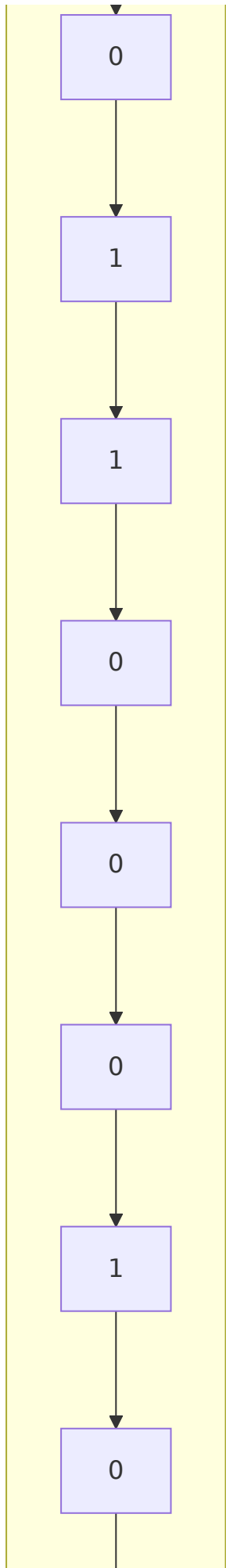
0

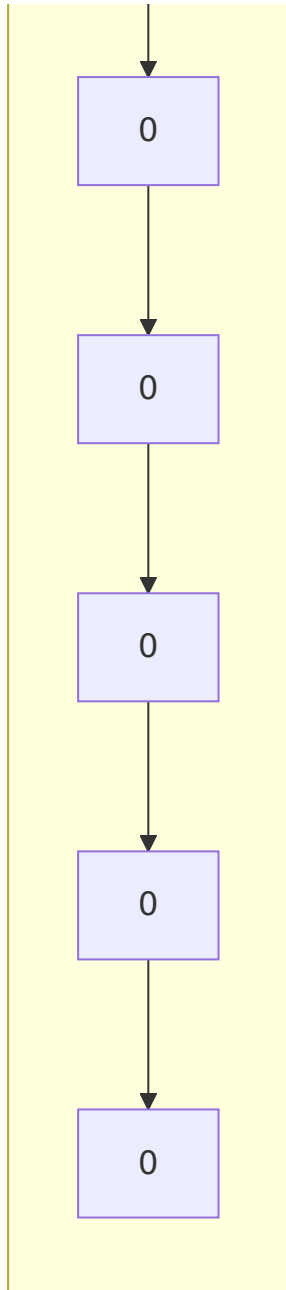
1











Scaling Weights in Neural Networks

- Each connection between an input node and a hidden layer node requires an estimated weight
 - These weights are estimated using backpropagation
- The number of weights scales with the size of the hidden layer
 - For a single hidden layer node, there are 36 connections (and thus 36 weights) from the 36 input nodes
 - Each additional node in the hidden layer adds an additional 36 weights that need to be estimated

Scalability Issues with Standard Neural Networks

- While standard networks work for very small images (like 6×6 pixels), they scale poorly as image size increases
- **[The Scaling Problem]** The number of weights required per node in the hidden layer is equal to the total number of pixels in the input image
 - For a 6×6 image, you estimate 36 weights per node
 - For a 100×100 image, you must estimate 10,000 weights per node
- This massive increase in weights makes the method inefficient for real-world, high-resolution pictures

Limitations of Standard Neural Networks

- **[Lack of Spatial Invariance]** Standard networks are sensitive to the exact position of features
 - If an image is shifted by even a single pixel, the network may fail to recognize the object correctly
 - Because the input nodes are tied to specific pixel locations, a shift changes the entire input vector
- **[Correlated Pixels]** Standard networks do not inherently account for the relationship between adjacent pixels
 - In complex images (like a photo of a teddy bear), pixels are highly correlated with their neighbors
 - Standard networks treat every input node as an independent entity, failing to capture these local patterns naturally

Introduction to Convolutional Neural Networks

- **[Exploiting Local Correlation]** Complex images contain patterns where adjacent pixels are highly related
 - For example, in a photo of a teddy bear, a brownish pixel is likely to be surrounded by other brownish pixels
 - Standard networks treat pixels as independent, but it is more effective to take advantage of these local correlations
- **Convolutional Neural Networks (CNNs)**
 - These are used for the classification of large and complicated images
 - They are designed to handle the scale and the spatial relationships that standard networks struggle with

How CNNs Make Image Classification Practical

Convolutional Neural Networks address the fundamental flaws of standard networks by performing three key functions:

1. Reducing the number of input nodes

- This helps solve the scalability problem where weight counts explode as image resolution increases

2. Tolerating small shifts

- The network can still recognize features even if the pixels are slightly moved or shifted within the image

3. Leveraging correlations

- CNNs take advantage of the fact that pixels in complex images are often highly correlated with their neighbors

The Role of Filters in CNNs

- **[The Convolution Process]** CNNs work by applying a filter to the input image
- **Filters (aka Kernels)**
 - These are small squares used to process the image
 - A common size for a filter is 3 pixels wide by 3 pixels tall
- **[Learning Filter Values]** The intensity of each pixel within the filter is not fixed
 - Before training, the filter starts with random pixel values

- During training, the values are optimized using backpropagation to become more useful for the specific task

Filter State	Example Values
Before Training	[[1, 0, 1], [0, 1, 1], [1, 1, 0]]
After Training	[[0, 0, 1], [0, 1, 0], [1, 0, 0]]

The Convolution Process

- **[Applying the Filter]** To process an image, the filter is overlaid onto a specific section of the input image
- **[The Mathematical Operation]** The process involves two steps:
 1. Multiply each overlapping pixel from the image by its corresponding pixel in the filter
 2. Add all those individual products together to reach a single final value
- **Dot Product**
 - This is the formal mathematical term for the "sum of products" calculation described above

Example Calculation:

Using a 3×3 filter overlaid on a section of the input image:

$$\begin{aligned}
 &(0 \times 0) + (0 \times 0) + (1 \times 1) \\
 &+ (0 \times 0) + (1 \times 1) + (0 \times 0) \\
 &+ (1 \times 1) + (0 \times 0) + (0 \times 0) \\
 &= 3
 \end{aligned}$$

The Convolution Operation

- **[Terminology]** Computing the **Dot Product** between the input and the filter is referred to as **convolving** the filter with the input
 - This specific operation is why they are called Convolutional Neural Networks
- **[Outputting Results]** After computing the dot product, a bias term is added to the result
- **Feature Map**
 - The final value (dot product + bias) is placed into a structure called a feature map

- **[The Sliding Process]** To process the entire image, the filter is slid over the input (e.g., moving one pixel at a time)

Filling the Feature Map

- **[The Iterative Process]** The calculation is repeated by shifting the filter across the input image
 - For each new position, calculate the dot product of the overlapping pixels
 - Add the bias term
 - Place the resulting value into the next available slot in the feature map
 - Continue this process until the entire feature map is filled
- **[Stride/Movement]** The distance the filter moves after each calculation can vary
 - In this example, the filter moves one pixel at a time
 - Other CNNs might move over 2 or more pixels at once

Summary of the Convolution Process

- **[The Workflow]** The process transforms an input image into a feature map through these steps:
 - Start with an input image (e.g., the letter "O")
 - Convolve a filter with that input
 - Add a bias term to the resulting values
 - Produce the final **Feature Map**
- **[The Benefit of Feature Maps]** They allow the network to leverage pixel correlations
 - Because each individual cell in a feature map corresponds to a group of neighboring pixels in the original image, the map captures how those pixels relate to one another spatially.

ReLU Activation Function

- **[Purpose]** Typically, the feature map is passed through a **ReLU (Rectified Linear Unit)** activation function
- **[Mechanism]** The function transforms the values based on their sign:
 - All negative values are set to **0**
 - All positive values remain exactly the same

ReLU Transformation Example

- Applying ReLU to a feature map containing both positive and negative values results in a sparse map where only positive signals persist.

Feature Map (Input)	Feature Map, Post ReLU
1, -1, -2, -1	1, 0, 0, 0
-1, -2, -1, -2	0, 0, 0, 0
-2, -1, -2, -1	0, 0, 0, 0
-1, -2, -1, 1	0, 0, 0, 1

- [Observation]** In this specific case, everything is set to zero except for the two positive points (the 1 s at the corners)

Subsequent Processing

- After the ReLU activation, the next step in the network is to apply another filter to this new, activated feature map.

Max Pooling

- [Definition]** A process where, instead of applying a filter through convolution, you simply select the **maximum value** within each region of the feature map
- [Mechanism]** The pooling filter typically moves across the map in a way that does not overlap itself

Max Pooling Example

Given a **Feature Map, Post ReLU**:

1	0	0	0
0	0	0	0
0	0	0	0
0	0	0	1

Applying Max Pooling results in a **Max Pooled** map:

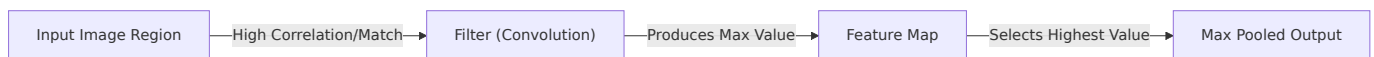
1	0
----------	----------

0	1
---	---

The Intuition Behind Max Pooling

- **[Connecting Features to Input]** Max Pooling reduces a region to the single spot where the filter performed its best match with the input image
 - A high value in the feature map indicates a strong correlation between the filter and that specific area of the input
 - The Max Pooling step selects this high value, effectively saying: "The feature we are looking for was found in this region"

Visualizing the Relationship



- **Example Summary:**
 - In the example, the upper left-hand corner of the input image has an exact match with the filter
 - This match results in the highest possible value in the feature map
 - Max Pooling then selects that specific spot, reducing the entire region to that one significant signal

Summary of Max Pooling

- **[Core Function]** Max Pooling selects the specific spots where the filter performed its best match with the input image
 - A region in the input image corresponds to a specific part of the feature map
 - Max Pooling preserves the single most significant signal from that region

Average or Mean Pooling

- **[Definition]** An alternative to Max Pooling where you calculate the **average value** for each region instead of selecting the maximum
- **[Mechanism]** While Max Pooling picks the "best" match, Mean Pooling provides a summary of the entire region's values

Transitioning to a Standard Neural Network

- **[Flattening]** Once the feature map has been pooled, it is converted into a column of input nodes

- This process "flattens" the spatial structure (the 2D grid) into a 1D vector
- This is necessary because standard neural networks expect a flat list of inputs rather than a grid
- **[The Final Classification Step]** The flattened input nodes are plugged into a standard neural network
 - In the provided example, the network consists of:
 - 4 Input Nodes (derived from the pooled layer)
 - A single Hidden Layer containing one node
 - The ReLU activation function applied to the hidden node

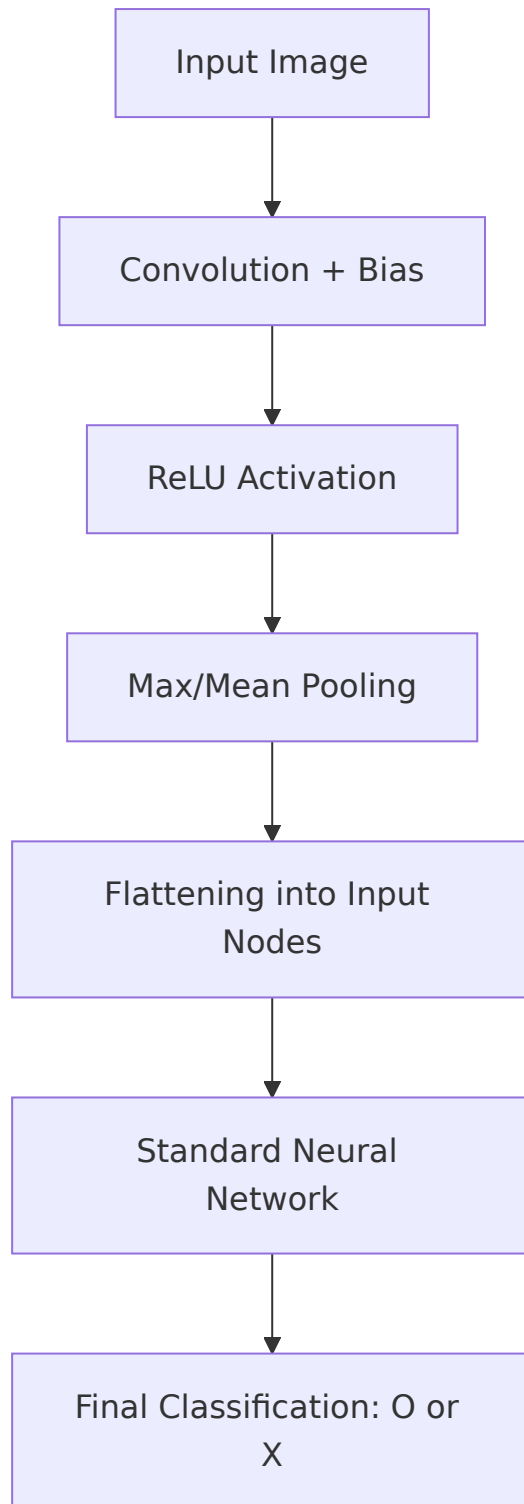
CNN Workflow Summary



Final Classification with a Standard Neural Network

- **[Network Architecture]** The flattened values from the pooling layer serve as the input for a standard neural network:
 - **Input Nodes:** 4 nodes (the values derived from the Max Pooling step)
 - **Hidden Layer:** A single hidden layer containing one node using the **ReLU activation function**
 - **Output Nodes:** 2 nodes representing the possible classes (e.g., 'O' and 'X')
- **[The Mathematical Step]** To calculate the hidden layer's value, the input nodes are multiplied by their associated weights and then summed:
 - For example, if an input node value is 1 and its weight is -0.8 , the product is $1 \times -0.8 = -0.8$
 - The network calculates these products for all input nodes and sums them up (plus a bias) to determine the signal sent to the output nodes

Complete CNN Pipeline Summary



Calculating the Hidden Layer Activation

- **[The Weighted Sum]** To find the value for a hidden node, multiply each input node by its corresponding weight, sum these products, and add the bias:
 - Using the example values:
 - $(1 \times -0.8) + (0 \times -0.07) + (0 \times 0.2) + (1 \times 0.17) + 0.97 = 0.34$

- **[Applying ReLU]** The resulting sum (0.34) acts as the x-axis coordinate for the activation function:
 - The ReLU function is defined as $f(x) = \max(0, x)$
 - Since $0.34 > 0$, the output (the y-axis coordinate) is also 0.34
 - $f(0.34) = \max(0, 0.34) = 0.34$
- **[Connection to Output]** This calculated activation value is then passed from the hidden layer to the output nodes to complete the classification (e.g., determining if the image is an 'O' or an 'X').

Final Classification Decision

- **[Output Node Connections]** The signal from the hidden layer is sent to the individual output nodes to determine the final class:
 - **Connection to 'O':** In this example, the connection results in a value of 1
 - **Connection to 'X':** The connection results in a value of 0
- **[Result]** Because the value for 'O' is higher than the value for 'X', the Convolutional Neural Network correctly classifies the input as a picture of the letter "O".

Processing a New Input (The Letter "X")

- **[Consistency of Learned Parameters]** Even when the input image changes, the core components of the trained model remain constant:
 - **The Filter:** The same convolutional filter used for the letter 'O' is applied to the new input.
 - **Max Pooling:** The pooling process and parameters remain identical.
 - **The Neural Network:** The weights and biases within the standard neural network layer do not change.
- **[The Pipeline Execution]** The model follows the same sequence of operations for the new input:
 1. Convolve the filter over the new input to generate a new feature map.
 2. Pass the feature map through the ReLU activation function.
 3. Perform Max Pooling on the activated feature map.
 4. Flatten the result to serve as input for the existing neural network.

Classification of the Letter "X"

- **[Input to Neural Network]** The values resulting from the Max Pooling layer are used as the input nodes for the standard neural network:
 - Input nodes: `[0, 1, 1, 0]`
- **[Neural Network Processing]** The neural network processes these inputs using its weights and biases:
 - **Connection to 'O':** The calculation results in 0
 - **Connection to 'X':** The calculation results in 1
- **[Result]** Because the value for 'X' is higher than the value for 'O', the CNN correctly classifies the input as the letter "X".

Benefits of Convolutional Neural Networks

- **[Input Reduction]** CNNs significantly decrease the number of input nodes needed for the subsequent standard neural network layer
 - In the example provided, a 6×6 image (36 potential inputs) was compressed down to just 4 inputs
- **[Leveraging Correlations]** CNNs take pixel correlations into account
 - This is achieved via the **Filter**, which examines a specific region of pixels at once, rather than processing each pixel in isolation

Spatial Invariance in CNNs

- **[Tolerating Pixel Shifts]** CNNs can handle small changes in the position of pixels within an image
 - This means the network can still recognize a pattern even if it is not in the exact same spot every time
- **[Demonstration of Shift Tolerance]** Shifting the letter "X" one pixel to the right still results in a successful classification:
 - Output value for "X": 1.23
 - Output value for "O": -0.2
 - Because 1.23 is much closer to 1 than -0.2 is, the network correctly identifies the shifted "X"

Enhancing Interpretability and Complexity

- **[Improving Output Interpretability]** To make raw output values easier to interpret, one can run them through specific functions:

- **SoftMax function:** Transforms outputs into a more readable format (e.g., probabilities).
- **Argmax function:** Selects the index of the highest value.
- **[Complexity Note]** The demonstrated architecture represents an extremely simplified version of a real-world Convolutional Neural Network.

Core Components of any CNN

- Regardless of complexity, all CNNs are built upon a specific sequence of operations:
 - **Convolution:** Applying filters to the input image
 - **Activation:** Applying an activation function to the output of the convolution
 - **Pooling:** Applying a pooling operation to the output of the activation function

